RESILPAY

RESILIENCE REPORT

Self-Healing Payroll System | Project Submission

Project	ResilPay — Self-Healing Payroll System
Application	Local academic demonstration
Technology	React, Python Flask, SQLite, Python healing logic
External dependency	Local mock payroll service on port 5001

1. Executive Summary

ResilPay is a demonstration payroll system designed to make service interruptions easier to detect, investigate, and recover from. It combines a React Payroll interface, a separate React Control Center, a Flask backend, an SQLite database, and Python healing logic that checks the health of a local mock payroll service.

The known network-healing path checks the dependency health endpoint, applies configured retry delays, and tracks recovery state. Incident records help administrators review failures. The goal is to demonstrate detection, retry-based recovery for a temporary service outage, and visibility into the result—not to claim every payment failure can be fixed automatically.

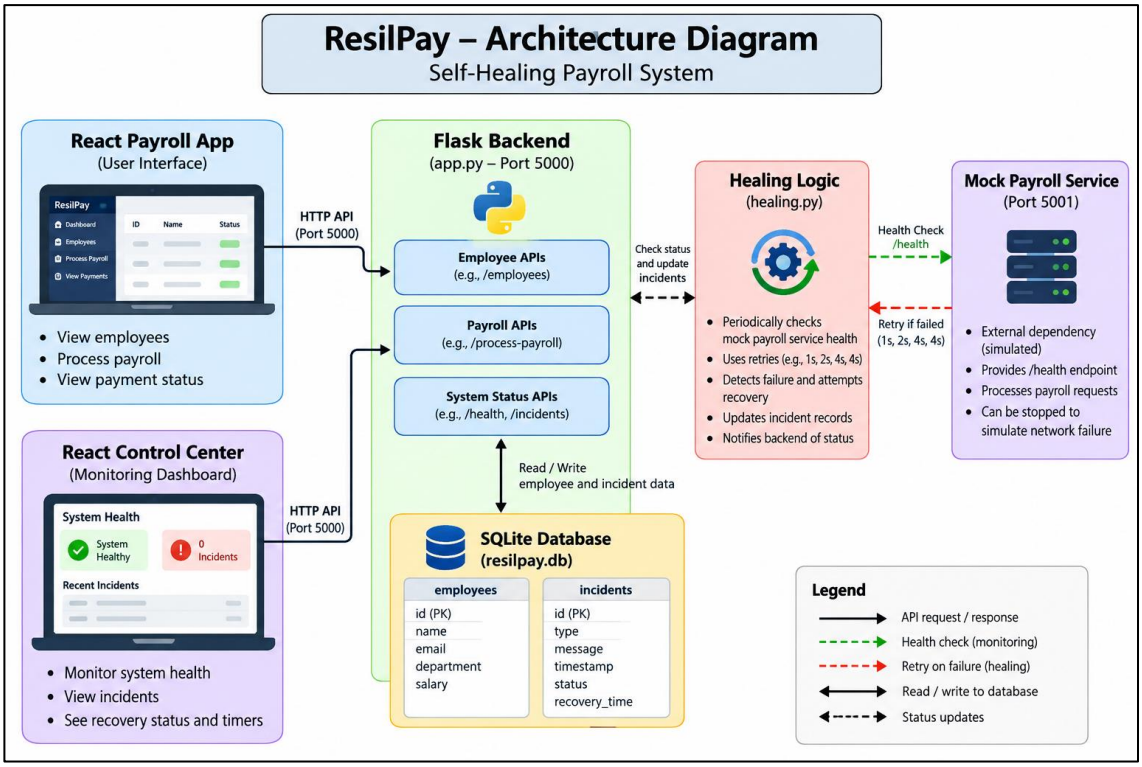
MEASUREMENT NOTE | Actual recovery-time results and verified before-and-after benchmarks were not provided. Values remain marked “Not measured” until recorded from real test runs.

Report Scope

This report describes the local project architecture and the known network-outage simulation. Database, API, and payment chaos cases are included as test-plan rows; claim successful recovery only after verifying the behavior in the running application.

2. Architecture & Resilience Design

The architecture diagram shows the major components and their responsibilities. It is a high-level view of the local demo, not a trace of every individual request.



Component Responsibilities

Component	Role in ResilPay
React Payroll app	Provides the user-facing payroll and employee views.
React Control Center	Presents system health, incidents, and recovery information.
Flask backend (app.py)	Serves application endpoints and status information on port 5000.
SQLite (resilpay.db)	Stores employee records and incident history in the local database.
Healing logic (healing.py)	Checks mock service health and attempts reconnection using configured retries.
Mock payroll service	Local dependency on port 5001; its health endpoint is used for network-failure testing.

Why These Mechanisms Were Selected

Health checks provide a concrete signal that the local dependency is responding. Bounded retries can recover from a short interruption without an immediate manual restart. State and incident tracking make the response visible to an administrator. These mechanisms are understandable, testable, and observable for a small prototype.

Known configuration: healing.py checks http://127.0.0.1:5001/health, uses a request timeout of 1 second, and configures retry delays of 1, 2, 4, and 4 seconds. Actual recovery time depends on failure duration, scheduling, and when the service becomes healthy again; it must be measured during a test.

3. Failure Handling & Test Evidence

The table separates the known network-healing path from scenarios that still need verification. Record recovery times from timed test runs; do not infer a time from retry configuration.

Failure	How detected	How it heals / responds	Measured recovery time
Network / mock service outage	Health request to /health fails or times out.	Configured retry mechanism attempts to reconnect; recovery state is tracked.	Not measured — record timed run.
Database unavailable	Detection method must be confirmed in the current build.	Do not claim automatic database repair unless implemented and tested. Record observed response.	Not verified — test and measure.
API / external request failure	Confirm which endpoint or simulated failure produces the signal.	Record actual retry, fallback, alert, or manual action observed.	Not verified — test and measure.
Payment failure / timeout	Use the payment test path and confirm how uncertain or failed status is represented.	May track status and require review; cannot force a bank or gateway to approve a rejected payment.	Not verified — test and measure.

How to Measure Recovery Time

1. Start the application and confirm the mock service is healthy.

2. Start a stopwatch immediately before injecting the failure and record the time.

3. Record when the failure first appears in the dashboard or incident log.

4. Record when the service is healthy again and the application confirms recovery.

5. Repeat each scenario at least three times and record the results.

Suggested definition: recovery time is the time from failure injection to the first confirmed healthy/recovered status. If detection time is also measured, report it separately.

4. Before-and-After Comparison

This table contains no invented values. Populate it after measuring the baseline/manual workflow and ResilPay workflow under comparable conditions.

Metric	Before / manual baseline	With ResilPay	Evidence to capture
Time to detect network outage	Not measured	Not measured	Injection timestamp and first detection.
Time to recover service	Not measured	Not measured	Injection timestamp and confirmed recovery.
Administrator effort	Not measured	Not measured	Time or steps needed to restore service.
Incident traceability	Record baseline process	Check actual incident record	Screenshot/export of incident and status.

5. Known Limitations

- Local simulation only: the mock payroll service is a local test dependency. This project does not demonstrate live integration with a real bank or payment gateway.
- Not every failure is recoverable: retries may help temporary availability issues, but cannot resolve every database fault, invalid input, rejected payment, or persistent external outage.
- Payment uncertainty requires care: if a request times out after submission, its outcome may be uncertain. A production system would need safe reconciliation and authoritative status checks before retrying.
- Some scenarios need further validation: database, API, and payment handling should be described as successful only after repeatable tests confirm the actual behavior.
- Recovery time varies: outage duration, timeout, retry timing, machine load, and UI refresh interval can affect observed results.

6. Future Improvements

Future work includes repeatable automated tests for each failure type, automatic recording of detection and recovery timestamps, safe payment-status checking, administrator notifications, and tests for persistent as well as temporary outages. A production version would also need stronger security, access controls, backups, and carefully managed payment-provider integration.

Conclusion: ResilPay demonstrates a practical resilience approach through health checks, bounded retries, recovery-state tracking, and incident visibility. Its effectiveness should be reported using actual test evidence, separating implemented behavior from future work.

ANNEXURE : AI USE DECLARATION

List every AI tool used while preparing the project. Review this declaration and edit it so it accurately reflects your own work before signing.

AI tool	What I asked it to do	What I wrote, changed or checked myself
ChatGPT	Help explain ResilPay concepts, plan the project presentation, troubleshoot development questions.	I reviewed and adapted the material for my project. I am responsible for checking and adding the code and features, verifying test results and recovery times, and ensuring I can explain every part of my submission.

Declaration: I confirm that this entry is my own work, that every AI tool used is listed above, and that I can explain every part of the submitted code.

Name: Khyati Sharma

Date: 27-10-2026

Video Title : **BCA3_khyatisharma_ Resilpay.mp4**